Assignment Task-by-Task Steps

How to Complete, Finalize, Deploy, and Submit NADRA LinkOps AI

This guide explains **each assignment requirement**, what it means, and exactly how to complete it using the project already created for you.

---

1. First understand one important thing

Your assignment does **not** require an Android app or desktop app only.

A **web app** also fully counts as an app if:

- it works properly,
- it solves a real problem,
- it includes an AI feature,
- it is uploaded to GitHub,
- and it is deployed at a public URL.

So **NADRA LinkOps AI is already the correct type of app** for this assignment.

What remains is to:

1. test it,
2. personalize it,
3. publish it,
4. deploy it,
5. submit it correctly.

---

2. Assignment Requirement 1 — YOUR OWN IDEA

What the teacher wants

They want an original problem from your real life, studies, workplace, or community.

How this project satisfies it

This app is based on your own real workflow:

- you work with NADRA network operations,
- you receive link complaints from regional sites,
- you check NMS,
- you contact telecom providers,
- you report to HQ,
- you track AD tasks.

What you must do now

Task 1.1 — Personalize the project language

Open `README.md` and make sure the introduction clearly says:

- this idea came from your real work context,
- it solves daily operational issues,
- it is for Quetta Region/NADRA network operations.

Task 1.2 — Be ready to explain it in viva

Memorize this short explanation:

> My project is NADRA LinkOps AI. It is a web app for a network engineer in NADRA Quetta Region. It helps manage connectivity complaints from NADRA sites, track disruption reports, classify genuine connectivity issues, handle telecom escalations, and also log AD support requests like OU transfer, password reset, and account unlock.

---

3. Assignment Requirement 2 — COMPLETE, FUNCTIONAL APP

What the teacher wants

Not just UI. The app must actually work end-to-end.

What must work in this project

- login screen
- dashboard
- complaint entry
- complaint register
- AD support entry
- reports
- AI assistant
- CSV export
- JSON backup/import

How to test each function

Task 2.1 — Test login

Use:

- `engineer@nadra.pk` / `Pass@123`

Check:

- login opens app,
- logout returns to login screen.

Task 2.2 — Test complaint entry

Go to **Complaints**.

Enter:

- site template
- incharge name
- provider

- issue type
- disruption start
- status

Click **Save Complaint**.

Check:

- record appears in complaint register,
- dashboard numbers update,
- report section later includes it.

Task 2.3 — Test complaint edit/delete

From complaint register:

- click **Edit**,
- change data,
- save again,
- then test **Delete**.

Task 2.4 — Test AD Support page

Go to **AD Support**.

Add a sample request such as:

- Employee: Ali Raza
- Type: Password Reset
- Status: Resolved
- Notes: Password refreshed and user informed

Save and verify it appears in the table.

Task 2.5 — Test Reports page

Go to **Reports**.

- choose the month,
- click **Generate Report**,
- click **Export Monthly CSV**,
- click **Print / Save PDF**.

Task 2.6 — Test AI Assistant

Go to **AI Assistant**.

- choose a complaint,
- optionally enter extra context,
- click **Draft AI Response**.

If no live key is connected, fallback text still demonstrates the feature.

Task 2.7 — Test backup/restore

In Reports page:

- click **Export Backup JSON**,

- then **Import Backup JSON**,
- verify the data can be restored.

---

4. Assignment Requirement 3 — AI-POWERED FEATURE

What the teacher wants

A real AI feature controlled by your own written instructions.

How this project satisfies it

The app already contains:

- custom AI system prompt,
- AI complaint classification,
- escalation email drafting,
- DAU update drafting,
- next-step recommendations.

Files involved

- `script.js`
- `api/ai-draft.js`
- `docs/system-prompt.md`

What you should do

Task 3.1 — Understand the AI feature

Open `docs/system-prompt.md`.

Read the system prompt so you can explain it to your teacher.

Task 3.2 — Make the AI live on deployment

Create an account on OpenRouter or another compatible provider.

Get an API key.

In Vercel environment variables, add:

- `OPENROUTER_API_KEY`
- `OPENROUTER_MODEL`
- `PUBLIC_APP_URL`

Example model:

- `openai/gpt-4.1-mini`

Task 3.3 — Test live AI after deployment

After redeploying on Vercel:

- open the live URL,
- use AI assistant,
- check whether live model responds.

---

5. Assignment Requirement 4 — BUILD IT YOURSELF

What the teacher wants

They allow any tools/framework, but the final app must be your own work.

How to present this honestly

You should say:

- the idea is yours,
- the workflow is yours,
- the problem context is yours,
- the app was developed for your real use case,
- and you understand every module.

What you should study before submission

Open these files and read them once:

- `index.html`
- `styles.css`
- `script.js`
- `api/ai-draft.js`
- `README.md`

You do not need to memorize every line, but you should know:

- what each file does,
- how complaints are stored,
- how reports are generated,
- how AI is called.

---

6. Assignment Requirement 5 — PUBLIC GITHUB REPOSITORY

What the teacher wants

A public GitHub repository that anyone can open.

How to do it

Task 5.1 — Create GitHub account

Go to:

- https://github.com

Create or sign in to your account.

Task 5.2 — Create repository

Create a new repository named:

- `nadra-linkops-ai`

Set it to:

- **Public**

Task 5.3 — Upload files
Upload the full project folder contents.
Important items include:

- `index.html`
- `styles.css`
- `script.js`
- `README.md`
- `vercel.json`
- `api/ai-draft.js`
- `assets/`
- `docs/`

Task 5.4 — If you use Git commands

```
cd nadra-linkops-ai
git init
git add .
git commit -m "Initial commit - NADRA LinkOps AI"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/nadra-linkops-ai.git
git push -u origin main
```

Task 5.5 — Verify repo is public
Open the repo link in **incognito/private browser mode**.
If it asks for login, it is not public yet.

---

7. Assignment Requirement 6 — DEPLOY LIVE AT A PUBLIC URL

What the teacher wants

The app must be live on a public URL.

Best option

Use **Vercel**.

How to do it

Task 6.1 — Create Vercel account
Go to:

- https://vercel.com

Sign in with GitHub.

Task 6.2 — Import your repo

- click **Add New Project**,
- choose `nadra-linkops-ai`,
- click **Import**,
- click **Deploy**.

Task 6.3 — Copy live URL

After deployment, Vercel gives a link like:

- `https://nadra-linkops-ai.vercel.app`

Copy that URL.

Task 6.4 — Add AI environment variables

Open project settings in Vercel and add:

- `OPENROUTER_API_KEY` = your real key
- `OPENROUTER_MODEL` = `openai/gpt-4.1-mini`
- `PUBLIC_APP_URL` = your deployed Vercel URL

Task 6.5 — Redeploy

After adding variables:

- redeploy the project,
- test the live app again.

---

8. Assignment Requirement 7 — README AS FULL REPORT

What the teacher wants

README must contain:

- app name,
- what it does,
- real problem solved,
- live URL,
- features,
- AI feature + prompt,
- tools/services/models,
- screenshots,
- how to run.

Good news

This project already has a strong README.

What you still must do

Task 7.1 — Replace placeholder GitHub URL

In `README.md`, replace:

- `https://github.com/your-username/nadra-linkops-ai`

with your real repo URL.

Task 7.2 — Replace placeholder live URL

In `README.md`, replace:

- `https://your-vercel-project.vercel.app`

with your real deployed link.

Task 7.3 — Recheck screenshots

The project already contains screenshots in:

- `assets/screenshots/`

If you want, you can replace them with screenshots from your final deployed app.

---

9. Submission Requirement — SUBMIT ONLY GITHUB LINK

What you must submit

Only this:

- your **public GitHub repository URL**

Example:

https://github.com/yourusername/nadra-linkops-ai

What not to submit

Do not submit:

- zip file,
- Google Drive link,
- private repo,
- only Vercel link,
- only screenshots.

---

10. If you want to complete the project by yourself, file by file

`index.html`

This controls:

- page structure,
- login page,
- tabs,
- forms,

- tables,
- reports UI.

`styles.css`

This controls:

- colors,
- typography,
- formal design,
- input field layout,
- cards,
- responsiveness.

`script.js`

This controls:

- login logic,
- tab switching,
- localStorage,
- complaint CRUD,
- AD request CRUD,
- dashboard calculations,
- reports,
- exports,
- AI frontend calls.

`api/ai-draft.js`

This controls:

- secure AI API request,
- reading environment variables,
- sending prompt + complaint context,
- returning AI output.

`README.md`

This is the main academic report.

`docs/`

This contains:

- tutorials,
- prompt docs,
- source notes,
- checklist,
- PDF guides.

---

11. Exact order you should follow now

Phase 1 — Local checking

1. Open the app.
2. Login.
3. Test complaint form.
4. Test AD form.
5. Test reports.
6. Test AI assistant.

Phase 2 — Personalization

7. Update README with your name/context if needed.
8. Replace any sample data you want to customize.
9. Take final screenshots if desired.

Phase 3 — Publishing

10. Create public GitHub repo.
11. Upload project files.
12. Verify repo in incognito.

Phase 4 — Deployment

13. Import GitHub repo into Vercel.
14. Deploy.
15. Add environment variables.
16. Redeploy.
17. Test live URL.

Phase 5 — Finalization

18. Replace placeholder links in README.
19. Re-test GitHub and live URL.
20. Submit only GitHub repo link.

---

12. Best practice checklist before submission

- [ ] Login works
- [ ] Complaint form works
- [ ] Complaint register works
- [ ] AD support works
- [ ] Reports work
- [ ] AI assistant works
- [ ] GitHub repo is public

- [ ] Live Vercel URL works
- [ ] README has real links
- [ ] No API keys committed
- [ ] App opens in incognito mode

---

13. Common mistakes students make

- repository is private,
- live link is broken,
- README is weak,
- AI feature is not explained,
- screenshots are missing,
- API key is committed in code,
- project idea is not clearly original.

Avoid all of these.

---

14. Final short answer for your teacher

If the teacher asks whether this is a complete app, you can answer:

> Yes. It is a complete deployed web app. It has login, complaint logging, reporting, AD support tracking, data export/import, and an AI feature for telecom escalation drafting. It is uploaded to a public GitHub repository and deployed on a public URL.

---

15. Final conclusion

This project already fits the assignment format.

You do **not** need to rebuild it into another type of app.

A deployed **web app** is fully valid.

Your real remaining work is:

- test it,
- publish it,
- deploy it,
- update the README,
- and submit correctly.